Phase 3 — Simulation Core

LineTwin · Accenture Innovation Challenge 2026 · Round 2 · Problem Track 4 "DigitalTwin.ai"

Purpose

Build the 30-station, 3-zone simulation core, carrying the two silent bugs the whole cascade thesis depends on getting right, and prove both regression tests actually test something rather than passing by coincidence.

A correction made before writing any code: parameter grounding without fabrication

The plan called for cycle-time and CV parameters "grounded" against Future Factories V2 and PyScrew. Before writing `scenarios/line30.yaml`, this was checked against what is actually available: this project has read **paper summaries** of those datasets via literature review, not the raw tables. No specific number in this codebase was ever extracted from either dataset.

Writing `source: future_factories_v2, mean_cycle_s: 42.3` for a number nobody actually computed from that data would have been exactly the fabrication `docs/CITATIONS.md` exists to ban — the difference between citing a real source and *laundering* an invented number through one.

Correction applied: every cycle-time, CV, and buffer-capacity value in `scenarios/line30.yaml` is stamped `synthetic — uncalibrated`. Future Factories V2 and PyScrew are cited only for the qualitative fact that real analogues with this general shape and variance structure exist — never as the source of a specific number. This is recorded directly in the YAML file's header comment, not only in this document, so the caveat travels with the config itself.

Two bugs, three guards, and how each was proven load-bearing

The merged station pattern (`src/twin/sim/station.py`) carries three `_unsafe_*` constructor flags, each defaulting to the safe behavior, each existing **only** so a test can reproduce its bug and prove the guard changes the outcome:

Guard	Bug it prevents	Proof it matters
Put issued while still occupied	**Bug A**: a fire-and-forget put lets a station accept new	`test_bug_a_guard_is_load_bearing` : safe path blocks 100% of runs measured; unsafe path blocks 0%

Guard	Bug it prevents	Proof it matters
	work before confirming the previous unit was accepted downstream — a slowdown never floods upstream, and the cascade thesis fails silently	
`.triggered` check before STARVED/BLOCKED	**Bug B**: setting state unconditionally truncates every active period to one state's duration	`test_bug_b_guard_is_load_bearing`: safe path merges a WORKING→DOWN→WORKING sequence into one 20s active period; unsafe path splits it into three
Active-period boundary only on ACTIVE↔INACTIVE transitions	Same failure mode as Bug B, tested independently at the state-machine level	`test_working_down_working_is_one_active_period` / `test_working_starved_working_is_two_active_periods`

A test-writing mistake was itself caught and corrected here. The first version of `test_bug_b_reproduction_incorrectly_splits_the_down_transition` asserted the unsafe path would produce two periods (`[(0,10), (10,20)]`). Running it showed the actual unsafe behavior produces **three** periods (`[(0,10), (10,15), (15,20)]`) — the unsafe flag closes the period on *every* transition, not just the DOWN one, which is a more complete demonstration of the bug than originally assumed. The test was wrong, not the implementation; fixed by correcting the expected value after inspecting the actual output rather than assuming the first guess was right.

Three real problems found and fixed while building this phase

1. A false-alarm "deadlock" that was actually a test-methodology error. An early manual run of the full 30-station line over 500 sim-seconds showed station S17 (the designated bottleneck) with zero completions — which looked like a bug. It was not: body-zone stations run ~45s cycle times, and with 16 stations upstream of S17, simple pipeline fill time is already ~700s before any unit can reach S17 at all. Re-run at 5000s showed the expected behavior immediately: S16 (immediately upstream of the bottleneck) accumulated real BLOCKED time, downstream stations accumulated STARVED time, and the cascade was visible exactly as designed. This is now documented directly in both `test_cascade.py` and `test_line.py` so nobody re-discovers the same false alarm — and both tests use a duration long enough that fill time cannot be mistaken for a defect.

2. A determinism test that failed for the right reason with the wrong diagnosis. The first version of the determinism test compared `units_completed` (a single small integer) between a seed and a different seed. Seed 2 and seed 1002 both produced exactly 9 completions over the test window — not because determinism was broken, but because an integer count over a short window is too coarse a signature to reliably distinguish two genuinely different random streams. Fixed by comparing the full list of sampled cycle times instead, which has enough resolution that a real difference cannot hide by coincidence. This is exactly the kind of failure that would be easy to "fix" by weakening the assertion rather than by strengthening the measurement — the fix here was the latter.

3. The source-agnosticism test was structurally unable to prove what it claimed. `test_simpy_...` asserted `"simpy" not in sys.modules` from inside an ordinary pytest test function. Because pytest runs every test file in one shared process, and `test_cascade.py` / `test_active_period.py` both import `twin.sim.station` (which imports `simpy`), the assertion was doomed to fail as soon as those files were collected in the same session — regardless of what `test_source_agnostic.py` itself imports. This is not a flaky test; it was checking a fact about process state that the test runner itself invalidates by design. **Fixed by moving the check into a genuinely isolated subprocess** (`subprocess.run([sys.executable, "-c", ...])`) that imports only `twin.sources` and nothing else, so no other test file's import graph can contaminate the result. The lesson generalizes: any "X was never imported" claim needs process isolation, not an in-process `sys.modules` check, the moment the test suite grows past one file.

Deliverables produced

Artefact	What it does
`scenarios/line30.yaml`	30 stations, 3 zones, mixed-model, 8 dark stations, with the parameter-provenance correction in its header
`src/twin/sim/dists.py`	`lognormal_params` — the sole sanctioned caller of `rng.lognormal`, empirically verified to recover the requested mean and CV, and to avoid the astronomical-value freeze from naive misuse
`src/twin/sim/rng.py`	Per-station `SeedSequence` → `PCG64` generators for Common Random Numbers
`src/twin/sim/station.py`	The merged station pattern with three `_unsafe_*` proof flags
`src/twin/sim/line.py`	Config-driven 30-station assembly with per-unit event logging against the Phase 2 frozen schema
`tests/test_dists.py`	3 tests
`tests/test_cascade.py`	5 tests (rewritten determinism test included)
`tests/test_active_period.py`	5 tests (corrected expected values)
`tests/test_line.py`	6 tests, full 30-station integration
`tests/test_source_agnostic.py`	Rewritten for genuine process isolation

Also landed this phase: the repo went live on GitHub

Mid-phase, at the user's request, the repository was pushed to GitHub as a **private** repo (`shreshtag30/linetwin`), which unblocked the CI-green gate Phase 2 had explicitly deferred. The first push failed CI immediately: `astral-sh/setup-uv@v10` does not exist as a resolvable tag — verified against the GitHub API directly, which showed only fully-qualified tags (`v10.0.0`, `v10.0.1`), no bare `v10` major-version alias. Re-pinned to `v10.0.1` and pushed again; **CI is now green on both ubuntu-latest and windows-latest**, ahead of schedule relative to the plan.

Exit criteria

Criterion	Status
Both regression tests green	Met
Both proven to fail red when their guard is removed	Met — all three `_unsafe_*` flags exercised and shown to change the observable outcome
Assertions on ranges, not exact tick-boundary counts	Met — no test asserts an exact count at a tick boundary
Parameter grounding honest about what is and isn't actually sourced	Met — corrected before shipping, not after
Full 30-station line integration tested, not just the 3-station primitive	Met — `test_line.py`, 6 tests
CI green on both operating systems	Met — ahead of the Phase 10 deadline the plan originally set for this

36/36 tests passing across the full suite (`tests/test_dists.py`, `test_cascade.py`, `test_active_period.py`, `test_line.py`, `test_fixture_matches_contract.py`, `test_source_agnostic.py` combined — Phase 2's fixture and source-agnosticism tests are included in this count, not additional to it).

Addendum: a real bug and a design gap, both found during Phase 4 groundwork

Before formalizing Phase 4's sensitivity harness, a quick probe perturbed each station's cycle time by ±15% and measured the effect on line throughput. Every non-bottleneck station showed **exactly** `+0.000` change — not "small," exactly zero, at every station tried. That result was suspicious enough to chase rather than accept, and it led to two real problems in already-committed Phase 3 code.

1. The variant multiplier was computed but never applied — a genuine bug. `Station.run()` called `self.cycle_time_sampler()` with zero arguments. The `part` object, which carried `variant_multiplier`, was never passed in. The line's "mixed-model" behavior — the entire premise of assigning sedan/SUV/ hatchback variants per unit — had no effect on simulated cycle times whatsoever from the moment `line.py` was first written. **Fixed**: `cycle_time_sampler` now takes the part as an argument, and `Station.run()` passes it through.

2. Even fixed, a single global scalar per variant cannot change which station is the bottleneck — a design gap, not a bug. A uniform multiplier scales every station identically, which preserves relative ranking by construction. Phase 4's requirement to produce "a labelled shifting-bottleneck trace" by sweeping variant mix would have been unsatisfiable no matter how the harness was written, because the scenario itself could never produce a shift. **Fixed**: variant multipliers are now per-zone (`variant_zone_multiplier[variant][zone]`), so an SUV-heavy mix can stress final assembly specifically, independent of body and paint.

3. Even with per-zone multipliers, the configured bottleneck was still an insurmountable ceiling — a tuning problem, verified rather than assumed fixed. At `bottleneck_multiplier=1.3` and `suv.final=1.35`, a 100%-SUV mix still only reached 74.2s at a final-assembly station against S17's 89.6s — never closing the gap. Retuned to `bottleneck_multiplier=1.15` and `suv.final=1.55`; rechecked the arithmetic (100% SUV: final 85.2s vs. S17 79.2s — now crosses), then **verified end-to-end in an actual simulation run**, not just in the static numbers: under the normal mix S17 leads (19,138s active vs. S13's 18,438s); under a heavy-SUV mix (90%), S19 genuinely overtakes S17 (18,777s vs. 18,656s). A new regression test, `test_variant_mix_can_genuinely_shift_the_bottleneck`, pins both directions so this capability cannot silently regress.

4. Writing that test surfaced a third variant of the pipeline-transient trap. The first version of the test used the module's usual 6000s duration and found `S01` — the very first station — "winning" the active-time comparison under the normal mix, not S17. This is a different failure mode from the two already documented in this phase (which were both about flow not having *arrived* yet): here, flow had reached every station, but comparing cumulative active time over a short fixed window is itself biased toward upstream stations, because they accumulate WORKING time from t=0 while downstream stations lose early time to STARVED periods during pipeline fill. Verified empirically that the bias washes out by 20,000s and fixed the test to use that duration for this specific comparison, with the reasoning documented inline so it isn't rediscovered a fourth time.

Net effect: the line now has a genuinely functioning mixed-model capability, a bottleneck that is dominant under normal conditions but not un-challengeable, and one more addition to this project's running list of "duration matters more than it looks like it should" traps. All 37 tests green (one more than the 36 reported above — the new shifting-bottleneck regression test).

Next

Phase 4 — Ground Truth by Sensitivity Analysis. Perturb each station's cycle-time mean by ±δ, measure the throughput response using the same per-station Common Random Numbers already built this phase, and define ground truth as the station with the largest sensitivity coefficient — the foundation the Phase 5 detector benchmark stands on, and something only possible because this project owns the simulator.